In [2]:
```
import numpy as np
import pandas as pd
import time
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_openml
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score,precision_score,recall_score,f1_score
```

In [3]:
```
print("Loading MNIST dataset...")

mnist = fetch_openml("mnist_784", version=1, as_frame=False)
X = mnist.data.astype("float32")
y = mnist.target.astype(int)

print("Dataset shape:", X.shape)
```

```
Loading MNIST dataset...
Dataset shape: (70000, 784)
```

In [4]:
```
X = X[:2000]
y = y[:2000]
```

In [5]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_stat
```

In [6]:
```
scaler=StandardScaler()
X_train=scaler.fit_transform(X_train)
X_test=scaler.transform(X_test)
```

In [7]:
```
# Sigmoid fn

model1 = MLPClassifier(
        hidden_layer_sizes=(128,64),
        activation='logistic',
        solver='adam',
        max_iter=1000,
        random_state=42)
```

In [8]:
```
# Relu fn

model2 = MLPClassifier(
        hidden_layer_sizes=(128,64),
        activation='relu',
        solver='adam',
        max_iter=1000,
        random_state=42)
```

In [9]:
```
# Tanh fn

model3 = MLPClassifier(
        hidden_layer_sizes=(128,64),
        activation='tanh',
        solver='adam',
```

```
        max_iter=1000,
        random_state=42)
```

In [10]:
```
# Model 1

#start timer
start_time1 = time.time()
#train
model1.fit(X_train,y_train)
#end timer
training_time1 = time.time()-start_time1
#predict
y_pred1 = model1.predict(X_test)
#accuracy
accuracy1 = accuracy_score(y_test,y_pred1)
```

In [11]:
```
# Model 2

#start timer
start_time2 = time.time()
#train
model2.fit(X_train,y_train)
#end timer
training_time2 = time.time()-start_time2
#predict
y_pred2 = model2.predict(X_test)
#accuracy
accuracy2 = accuracy_score(y_test,y_pred2)
```

In [12]:
```
# Model 3

#start timer
start_time3 = time.time()
#train
model3.fit(X_train,y_train)
#end timer
training_time3 = time.time()-start_time3
#predict
y_pred3 = model3.predict(X_test)
#accuracy
accuracy3 = accuracy_score(y_test,y_pred3)
```

In [13]:
```
print("Sigmoid function:-")
print("Accuracy:", accuracy1, "| Trainig Time:", training_time1)
print("\n")
print("ReLu function:-")
print("Accuracy:", accuracy2, "| Trainig Time:", training_time2)
print("\n")
print("Tanh function:-")
print("Accuracy:", accuracy3, "| Trainig Time:", training_time3)
```

```
Sigmoid function:-
Accuracy: 0.8625 | Trainig Time: 2.809378147125244


ReLu function:-
Accuracy: 0.895 | Trainig Time: 0.7686443328857422
```

```
Tanh function:-
Accuracy: 0.87 | Trainig Time: 1.440636157989502
```

In [14]:

```
loss_curve = {}
loss_curve["logistic"] = model1.loss_curve_
loss_curve["relu"] = model2.loss_curve_
loss_curve["tanh"] = model3.loss_curve_
```

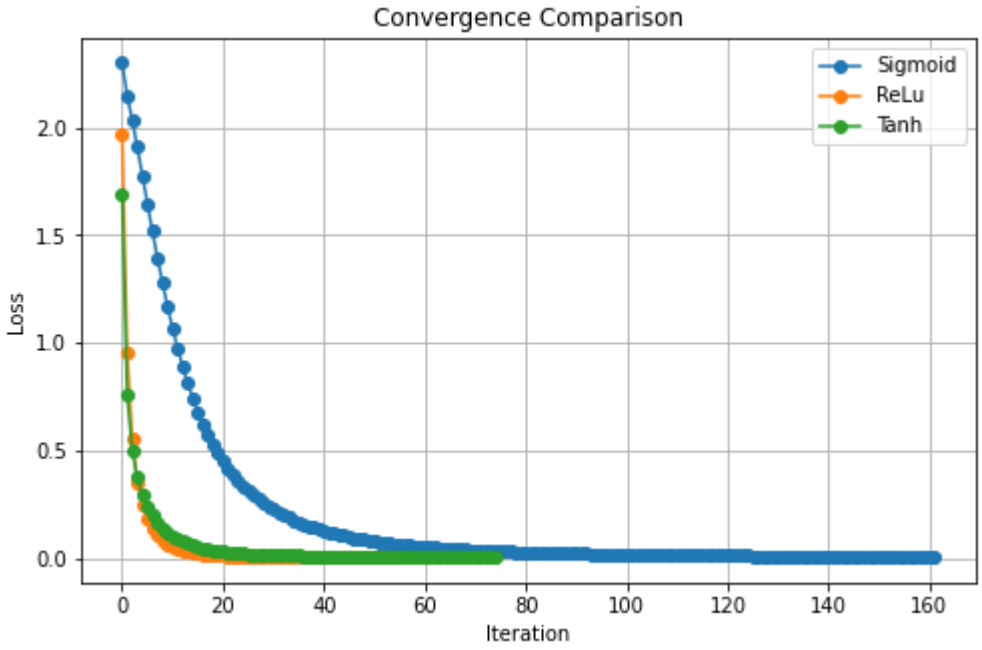
In [15]:

```
plt.figure(figsize=(8,5))

plt.plot(loss_curve["logistic"], marker="o", label="Sigmoid")
plt.plot(loss_curve["relu"], marker="o", label="ReLu")
plt.plot(loss_curve["tanh"], marker="o", label="Tanh")

plt.xlabel("Iteration")
plt.ylabel("Loss")

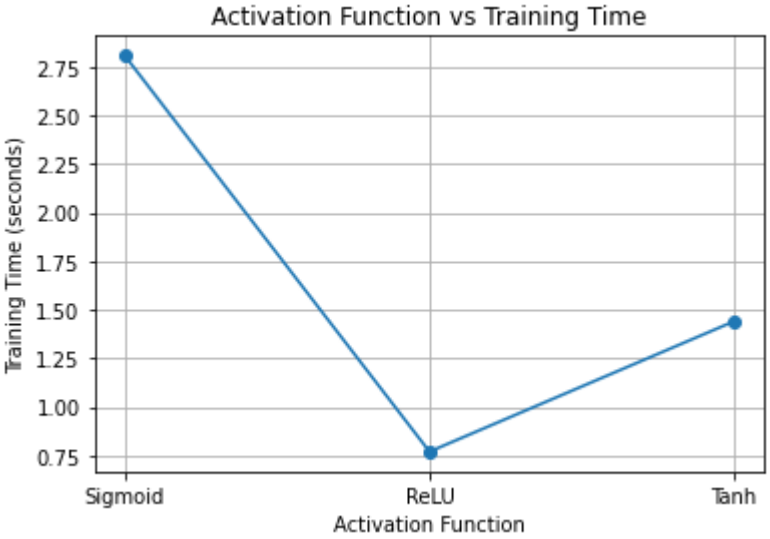
plt.title("Convergence Comparison")
plt.legend()
plt.grid()
```



In [20]:

```
plt.plot(['Sigmoid', 'ReLU', 'Tanh'], [training_time1, training_time2, training_time

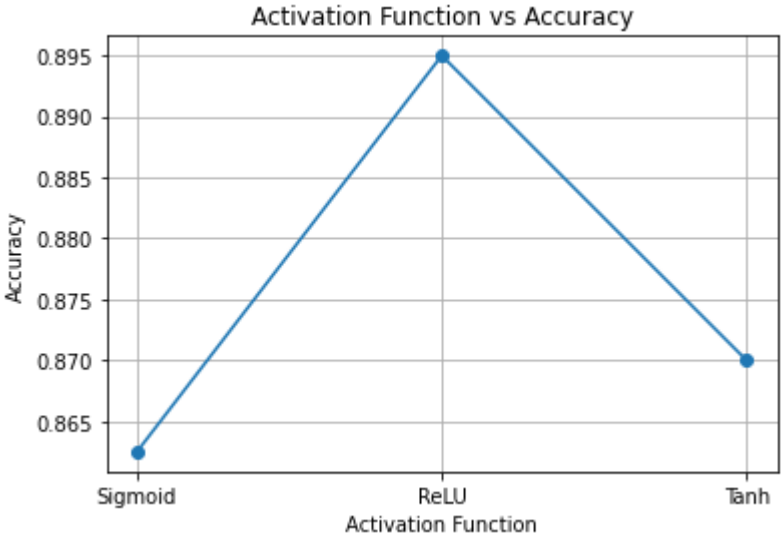
plt.xlabel("Activation Function")
plt.ylabel("Training Time (seconds)")
plt.title("Activation Function vs Training Time")
plt.grid(True)
plt.show()
```



In [22]:

```
plt.plot(['Sigmoid', 'ReLU', 'Tanh'], [accuracy1, accuracy2, accuracy3], marker='o')

plt.xlabel("Activation Function")
plt.ylabel("Accuracy")
plt.title("Activation Function vs Accuracy")
plt.grid(True)
plt.show()
```



In [ ]: